

SELF-BALANCING ROBOT

Technical Documentation & Build Guide

Arduino UNO • MPU6050 • L298N • DC Gear Motors • PID Control

Project Type	Two-wheeled self-balancing robot
Controller	Arduino UNO
IMU	MPU6050 6-axis accelerometer + gyroscope
Control Method	Complementary filter + PID feedback

Prepared as a practical reference for assembly, programming, calibration, PID tuning, testing, and troubleshooting.

Table of Contents

1. Project Overview
2. Objectives and Working Principle
3. Hardware Components
4. Pin Connections and Wiring
5. Mechanical Construction
6. Power System and Grounding
7. MPU6050 Sensor and Angle Estimation
8. Complementary Filter
9. PID Control System
10. Motor Control
11. Arduino Program Structure
12. Current PID Tuning Configuration
13. Recommended PID Tuning Procedure
14. Serial Monitoring and Live Ki Adjustment
15. Calibration Procedure
16. Testing Procedure
17. Troubleshooting Guide
18. Common Failure Modes
19. Safety Precautions
20. Future Improvements
21. Project Summary

1. Project Overview

A self-balancing robot is an inverted-pendulum control system. The robot has two driven wheels and must continuously move those wheels underneath its center of mass to prevent the chassis from falling. The MPU6050 measures the robot's orientation and angular motion, the Arduino UNO processes the sensor data, and a PID controller converts the estimated tilt error into a motor command.

The project described in this document uses a complementary filter to estimate pitch angle from the MPU6050 and a PID feedback loop to control two DC motors through an L298N motor driver.

2. Objectives and Working Principle

Primary objectives:

- Measure the robot's tilt angle using an MPU6050.
- Combine accelerometer and gyroscope measurements to obtain a stable angle estimate.
- Calculate the difference between the desired upright angle and the measured angle.
- Use PID control to determine the direction and magnitude of motor correction.
- Drive both motors in the required direction through the L298N.
- Tune the system so that the robot recovers from small disturbances without excessive oscillation.

Control loop

1. MPU6050 → raw accelerometer and gyroscope data
2. Angle estimation → complementary filter
3. Error calculation → setpoint – measured angle
4. PID controller → proportional + integral + derivative correction
5. Motor driver → PWM and direction commands
6. Motors → robot motion
7. Robot motion → new sensor measurements

This is a closed-loop control system: the output motion changes the measured tilt, and the controller continuously updates the motor command based on the new measurement.

3. Hardware Components

Component	Purpose	Notes
Arduino UNO	Main controller	Reads MPU6050 and generates motor PWM/direction signals.
MPU6050	Tilt and angular-rate sensing	3-axis accelerometer + 3-axis gyroscope.
L298N	Dual H-bridge motor driver	Controls two DC motors; ENA/ENB provide PWM speed control.
2 × DC gear motors	Robot propulsion	Prefer matched motors with similar gearing.
2 × wheels	Locomotion	Rigidly coupled to the motor shafts.
Robot chassis	Mechanical structure	Keep the center of mass close to the wheel axle.
Battery / power source	Motor power	Voltage must match the motor and driver requirements.
5 V buck converter	Logic power	Use a stable regulated output for Arduino/sensors.
Jumper wires / connectors	Electrical connections	Keep sensor and motor wiring mechanically secure.

4. Pin Connections and Wiring

MPU6050 → Arduino UNO

MPU6050 Pin	Arduino UNO	Function
VCC	5V (for common breakout boards that include regulation)	Power
GND	GND	Common ground
SDA	A4	I ² C data
SCL	A5	I ² C clock

L298N → Arduino UNO

L298N / Motor Driver	Arduino Pin	Function
ENA	D5	PWM speed control, Motor A
IN1	D4	Motor A direction
IN2	D3	Motor A direction
ENB	D6	PWM speed control, Motor B
IN3	D7	Motor B direction
IN4	D8	Motor B direction
GND	Arduino GND	Common ground
OUT1 / OUT2	Motor A	Motor A terminals
OUT3 / OUT4	Motor B	Motor B terminals

Important: the exact motor direction depends on the motor wiring. If the robot moves in the wrong direction when it should correct itself, reverse the motor polarity or invert the software direction logic.

5. Mechanical Construction

- Mount the two motors so their shafts are aligned on the same horizontal wheel axis.
- Mount the MPU6050 rigidly to the chassis. Avoid loose mounting, foam mounting, or excessive vibration.
- Keep the sensor orientation consistent with the axis used in the software.
- Keep the center of mass slightly above the wheel axle; an excessively tall or excessively low geometry changes the dynamics.
- Keep the left and right wheels similar in diameter and traction.
- Minimize mechanical backlash and loose couplings.
- Ensure both wheels contact the floor evenly.

A robot that is mechanically asymmetric can be difficult to tune even when the code is correct. Before changing PID values, verify that the chassis is rigid and both motors behave similarly.

6. Power System and Grounding

Motor current creates electrical noise and voltage transients. The motor supply should therefore be kept separate from the regulated logic supply where practical, while maintaining a common ground reference.

- Battery → L298N motor supply.
- Battery → buck converter → regulated logic supply.
- Arduino GND, MPU6050 GND, L298N GND and buck converter GND must share a common ground.
- Do not feed an unknown or unstable voltage into the Arduino.
- A regulated 5 V rail is appropriate for the Arduino 5 V pin and typical MPU6050 breakout boards; if using VIN, follow the board's recommended input range instead.
- Check battery voltage under load, not only with the robot switched off.

7. MPU6050 Sensor and Angle Estimation

The accelerometer can estimate the gravity vector and therefore provide an absolute tilt reference at low dynamic acceleration. The gyroscope measures angular velocity and responds quickly, but its integrated angle drifts because of bias. A self-balancing robot needs both characteristics.

```
float accAngle = atan2(AcY, AcZ) * 180.0 / 3.14159265;
float gyroRate = GyX / 131.0;
```

For the ± 250 °/s MPU6050 gyroscope setting, 131 LSB/(°/s) is the commonly used scale factor. The axis selection must match the physical orientation of the MPU6050 on the robot.

8. Complementary Filter

The complementary filter combines the fast gyroscope estimate with the slower but drift-resistant accelerometer angle. The current recommended form for this project is:

```
anglePitch = 0.96 * (anglePitch + gyroRate * elapsedTime)
             + 0.04 * accAngle;
```

The 0.96 coefficient gives the gyro estimate more short-term influence, while 0.04 gives the accelerometer a continuous correction against long-term drift. If the angle is noisy, the accelerometer contribution may need to be reduced. If the angle slowly drifts, it may need to be increased.

9. PID Control System

The controller compares the desired upright angle (setpoint) with the measured pitch angle. The resulting error is used by three terms:

Term	Role	Typical effect
P — Proportional	Responds to current tilt error.	Higher Kp gives stronger correction but can cause oscillation.
I — Integral	Accumulates persistent error.	Removes long-term bias but can cause windup and slow oscillation.
D — Derivative	Responds to the rate of change of error.	Adds damping and reduces overshoot, but is sensitive to noise.

For this project:

```
error = setpoint - anglePitch;

cumError += error * elapsedTime;
cumError = constrain(cumError, -10, 10);

float derivative = (error - lastError) / elapsedTime;
filteredD = 0.8 * filteredD + 0.2 * derivative;

lastError = error;

pidOutput = (Kp * error)
            + (Ki * cumError)
            + (Kd * filteredD);
```

10. Motor Control

The PID output is constrained to the Arduino PWM range of approximately -255 to +255. The sign controls direction and the magnitude controls PWM duty cycle.

```
pidOutput = constrain(pidOutput, -255, 255);
```

A minimum PWM offset is used to overcome static friction. The current project uses a starting value of 30. This value should be experimentally determined because it depends on the motor, gearbox, battery, driver losses, wheel load, and floor surface.

```
const int MIN_MOTOR_PWM = 30;
```

11. Arduino Program Structure

8. Initialize Serial communication and I²C.
9. Wake the MPU6050 from sleep mode.
10. Configure motor-driver pins as outputs.
11. Measure elapsed time using micros().
12. Read 14 bytes from the MPU6050.
13. Calculate accelerometer angle and gyro rate.
14. Update the complementary-filter angle estimate.
15. Calculate PID error, integral and filtered derivative.
16. Limit PID output to safe PWM range.
17. Stop the motors if the robot falls beyond the safety angle.
18. Drive both motors in the required direction.
19. Print diagnostic values periodically.

12. Current PID Tuning Configuration

Parameter	Current starting value	Purpose
Kp	20.0	Main corrective response
Kd	1.3	Damping / anticipatory correction
Ki	0.0	Start disabled during initial tuning
Setpoint	0.0°	Nominal upright position; may require calibration
MIN_MOTOR_PWM	30	Starting motor deadband compensation
Filter	0.96 gyro / 0.04 accelerometer	Angle estimation
Fall cutoff	±40°	Motor safety cutoff

These values are starting points, not universal values. The correct values depend strongly on wheel diameter, motor torque/speed, chassis mass, center-of-mass height, battery voltage, gearing, and floor friction.

13. Recommended PID Tuning Procedure

Tune the controller in this order: Kp → Kd → setpoint → Ki.

20. Set Ki = 0. Do not tune integral first.
21. Hold the robot upright by hand or use a support so the motors can respond safely.
22. Increase Kp gradually. The robot should respond more strongly as it leans.
23. If the robot oscillates rapidly, reduce Kp or increase Kd slightly.
24. Increase Kd gradually to damp oscillations.
25. Check whether the robot naturally balances at a non-zero angle. Adjust setpoint in small increments, such as 0.2°–0.5°.
26. Only after P and D are stable, introduce a small Ki value such as 0.1, 0.2, 0.5, etc.
27. If the robot slowly drifts in one direction because of persistent bias, increase Ki cautiously.

28. If the robot develops slow oscillations or becomes unstable after several seconds, reduce K_i and clear the integral term.

A useful symptom-to-action guide:

Observed behavior	Likely adjustment
Robot reacts too slowly and falls before correcting	Increase K_p .
Fast left-right oscillation	Reduce K_p or increase K_d slightly.
Robot overshoots after a tilt	Increase K_d slightly.
Robot is stable but slowly leans one way	Adjust setpoint; then consider small K_i .
Robot becomes unstable after several seconds	Reduce K_i ; check gyro bias and integral windup.
Motors buzz but do not move	Increase minimum PWM carefully or check motor driver/power.
Robot shoots away immediately	Check motor direction and sensor axis/sign before PID tuning.

14. Serial Monitoring and Live K_i Adjustment

The program can accept a command such as $i0$, $i0.2$ or $i1$ through the Serial Monitor to change K_i while running.

```
i0
i0.2
i0.5
i1.0
```

Use Serial Monitor at 115200 baud. The program resets the accumulated integral when K_i is changed.

Diagnostic output includes:

```
Angle: ... | Kp: ... | Kd: ... | Ki: ... | PWM Output: ...
```

15. Calibration Procedure

15.1 Mechanical zero / setpoint

29. Place the robot on a level surface.
30. Make sure the MPU6050 is firmly mounted.
31. Determine the angle at which the robot is mechanically closest to a stable upright position.
32. If this is not 0° , change setpoint by small increments.

15.2 Gyroscope bias

When the robot is completely stationary, the gyro should ideally read close to zero. A constant offset can cause angle drift. A more advanced version of the program should collect several hundred samples at startup and calculate gyro bias before enabling the balancing loop.

15.3 Sensor axis verification

The current code uses AcY with AcZ and GyX . These axes are correct only if the MPU6050 is mounted in the corresponding orientation. If the sensor is rotated, the software must use the matching accelerometer and gyro axes and possibly invert their signs.

16. Testing Procedure

33. Test the Arduino and MPU6050 first without motors.

34. Confirm the angle changes smoothly when the robot is tilted by hand.
35. Confirm that the angle returns toward the correct reference when the robot is returned upright.
36. Test each motor independently.
37. Confirm the motor direction needed for a forward/backward balancing correction.
38. Raise the robot so the wheels are off the floor and test low PWM.
39. Set a conservative fall cutoff and keep a hand near the power switch.
40. Perform the first floor test with $K_i = 0$.
41. Increase K_p and K_d gradually.
42. Only introduce K_i after stable P+D behavior is achieved.

17. Troubleshooting Guide

Problem	Possible cause	Check / solution
Angle drifts slowly	Gyro bias / insufficient accelerometer correction	Calibrate gyro and review complementary-filter coefficients.
Angle jumps or is noisy	Mechanical vibration / noisy derivative	Rigidly mount MPU6050 and use filtered derivative.
Robot moves in the wrong direction	Motor polarity or PID sign is reversed	Verify correction direction before tuning PID.
Robot balances briefly then falls	Low K_p , drift, wrong setpoint, motor mismatch, or battery sag	Check angle drift, setpoint, motor response and supply voltage.
Motors are too aggressive	K_p or minimum PWM too high	Reduce K_p / MIN_MOTOR_PWM.
Robot oscillates quickly	Excessive K_p or insufficient damping	Reduce K_p and/or increase K_d slightly.
Slow oscillation develops	Integral too high	Reduce K_i and clear integral.
One wheel is stronger	Motor mismatch / mechanical asymmetry	Check wiring, gearbox, wheel friction and motor calibration.
Arduino resets	Power rail disturbance / motor noise	Use a stable regulator, common ground, good wiring and adequate decoupling.
MPU6050 values are invalid	I ² C wiring or power issue	Check SDA=A4, SCL=A5, VCC and GND; verify I ² C communication.

18. Common Failure Modes

A. Robot balances for a few seconds and then slowly falls

This behavior commonly points toward angle drift, a slightly incorrect setpoint, insufficient correction authority, motor mismatch, or accumulated integral error. First run with $K_i = 0$ and observe the reported angle. If the reported angle changes while the stationary robot is actually still, investigate gyro bias and sensor fusion.

B. Robot immediately drives away

Do not attempt to fix this by randomly changing K_p . First verify the feedback sign. If the robot tilts forward, the wheels must move in the direction that brings the wheel axle underneath the center of mass. If the controller commands the opposite direction, reverse the motor direction logic or the sensor sign.

C. Robot vibrates continuously

This can be caused by excessive K_p , noisy derivative calculation, mechanical vibration, high minimum PWM, or a loop that is too slow. Filter the derivative and verify the MPU6050 mounting.

19. Safety Precautions

- Keep fingers away from wheels and motor shafts during powered tests.
- Use a quick-disconnect or easily accessible power switch.
- Test the robot with the wheels lifted before the first floor test.
- Do not leave the robot unattended while powered.
- Verify battery polarity and voltage before connecting electronics.
- Do not connect an unstable buck-converter output to the Arduino.
- Use the fall cutoff to stop the motors when the robot exceeds a safe tilt angle.
- Secure loose wires so they cannot contact moving wheels.

20. Future Improvements

- Automatic MPU6050 gyro-bias calibration at startup.
- Automatic setpoint calibration.
- PID parameter storage in EEPROM.
- Independent left/right motor compensation.
- Encoder feedback for wheel speed and position control.
- Higher-performance motor driver with lower voltage drop than the L298N.
- Battery-voltage monitoring and low-voltage protection.
- Moving-average or more advanced filtering where appropriate.
- Non-blocking loop timing instead of `delay(10)`.
- A cascaded controller with inner angle loop and outer velocity/position loop.
- Bluetooth or Wi-Fi telemetry for real-time PID tuning.
- OLED dashboard for angle, PID values, battery voltage and motor output.

21. Project Summary

The self-balancing robot is a practical demonstration of embedded systems, robotics, sensor fusion, feedback control, motor control and real-time programming. Its core challenge is not simply making the motors move; it is maintaining a stable closed-loop relationship between the robot's physical tilt and the corrective motor torque.

For the current implementation, the recommended development sequence is: verify mechanical construction → verify MPU6050 axes and angle → verify motor correction direction → calibrate sensor bias → tune Kp → tune Kd → adjust setpoint → introduce Ki cautiously. This sequence minimizes the chance of compensating for a hardware or sensor problem with inappropriate PID values.

Appendix A — Key Code Sections

```
// PID settings
const float Kp = 20.0;
const float Kd = 1.3;
float Ki = 0.0;

float setpoint = 0.0;
float error = 0;
```

```

float lastError = 0;
float cumError = 0;
static float filteredD = 0;

// Complementary filter
float accAngle = atan2(AcY, AcZ) * 180.0 / 3.14159265;
float gyroRate = GyX / 131.0;

anglePitch = 0.96 * (anglePitch + gyroRate * elapsedTime)
             + 0.04 * accAngle;

// PID
error = setpoint - anglePitch;

cumError += error * elapsedTime;
cumError = constrain(cumError, -10, 10);

float derivative = (error - lastError) / elapsedTime;
filteredD = 0.8 * filteredD + 0.2 * derivative;

lastError = error;

pidOutput = (Kp * error)
            + (Ki * cumError)
            + (Kd * filteredD);

pidOutput = constrain(pidOutput, -255, 255);

```

Appendix B — Project Test Record

Use this table to record your actual tuning results.

Test	Kp	Kd	Ki	Setpoint	Result / observation
1					
2					
3					
4					
5					
6					

End of Documentation